Daffodil International University
Department of Software Engineering

Project Proposal

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

Submitted by

S. M. Nihal Ahmed	ID: 232-35-002
Afrim Hossen Khan	ID: 232-35-076
Sabikun Nahar Sinthia	ID: 232-35-475

BSc in Software Engineering
Daffodil International University (DIU)

Submitted for: Capstone Project Approval

Supplementary document: Market Analysis of Bikroy.com and Its Alternatives

October 2026

Contents

1 Introduction **2**
1.1 Background Overview . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 3
1.4 Scope . . . 4
1.5 Stakeholders . . . 4
1.6 Proposed Solution . . . 4
1.7 Game-Account Escrow . . . 5
1.8 AI Features . . . 6

2 System Requirements **7**
2.1 Functional Requirement Areas . . . 7
2.2 Key Non-Functional Requirements . . . 7
2.3 Acceptance Criteria . . . 8

3 Tools and Technologies **8**
3.1 Backend . . . 8
3.2 Data Storage . . . 9
3.3 Real-Time Communication . . . 9
3.4 Frontend . . . 9
3.5 External Integrations . . . 9
3.6 Infrastructure, Testing and Quality . . . 10

4 Project Timeline and Work Plan **10**
4.1 Development Approach . . . 10
4.2 Team and Roles . . . 11
4.3 Timeline (12 Weeks / 3 Months) . . . 11
4.4 Risk Management . . . 12

1 Introduction

1.1 Background Overview

Second-hand goods change hands constantly in Bangladesh, but almost entirely through unmoderated Facebook groups and general classifieds boards such as Bikroy. None of these venues solve the problem that actually makes resale between strangers hard: nobody vouches for either side, nobody holds the money until the item is confirmed delivered, and if a seller wants to run a genuine auction rather than a fixed price, there is no mechanism to stop last-second sniping, fake bidding, or a buyer who simply vanishes after "winning." Delivery is negotiated ad hoc over chat, with no fare transparency and no fallback if the courier or the buyer disappears.

The market involved is large. Bikroy, the largest classifieds board, has operated since 2012 and carries about 297,000 live ads, most of them used phones, electronics and household goods, with about 400,000 monthly buyers [1, §2.3–2.4]. Yet its own 2014 audit found only 66% of sampled ads genuine, and no later audit has been published [1, §2.1].

Why Existing Platforms Don't Solve This

It is worth being specific about why the obvious alternatives fall short, since the case for building something new rests on it.

Facebook Marketplace and the buy-and-sell groups around it offer no identity verification beyond a social login and no payment handling at all [1, §5.2]. They also have no real auction mechanic: "highest offer in the comments" has no enforced end time and leaves no record of who actually committed to buy, so a seller who tries to auction an item has nothing to hold the winner to.

Bikroy and similar classifieds boards are listing directories, not transaction platforms. They connect a buyer and a seller and then step out of the way, so whatever happens with payment and delivery happens entirely off-platform, with no escrow and no dispute mechanism. Bikroy's own terms make this explicit: it states that it is not a party to transactions between users and caps its liability at BDT 6,700 [1, §2.1].

Daraz and other e-commerce marketplaces do offer payment and delivery infrastructure [1, §5.3], but they are built for businesses selling new inventory at scale, not for one individual selling one used item to another. There is no auction format, and the seller-verification bar is built around business registration rather than personal KYC.

None of these platforms combine identity verification, escrow, fair auctions, and delivery coordination into a single flow for individual resale [1, §5.5], which is precisely the space TrustBazaar is meant to occupy.

1.2 Problem Statement

A buyer and seller who have never met are being asked to trust each other with real money and an item, with no institution standing behind either promise. Four consequences follow directly from that, and each is a concrete reason informal channels fail in practice.

The first is **no accountability**. Anyone can create a profile and list an item, and nothing ties a listing to a real, verifiable person, so a defrauded buyer has no recourse and a scammer can simply reappear under a new profile. Bikroy's phone-number check, for example, is defeated by registered SIM cards bought for Tk 350–500 [1, §3.2].

The second is **no payment safety**. Money changes hands either before or after delivery, with no neutral holding point in between, so either the buyer risks paying for an item that never arrives

or the seller risks shipping an item that is never paid for. Advance payment to fake sellers is one of the scam patterns documented on Bikroy, and the escrow rule Bangladesh Bank introduced for e-commerce in 2021 does not reach person-to-person deals [1, §3.2, §3.5].

The third is **no fair price discovery**. Where sellers do attempt informal "highest offer wins" auctions in comment threads, there is no enforced end time, no protection against a bid placed in the final second, and no verifiable record of who actually won, so the seller has no way to hold the winning bidder to the deal.

The fourth is **no safe way to trade game accounts**. Accounts for games such as PUBG Mobile, Free Fire, Clash of Clans and Steam cannot be inspected like a physical item. They are traded through Facebook-group "admin deals", in which one unverified person holds both the buyer's money and the seller's password, and even after an honest exchange the seller can recover the account after the sale [1, §5.2].

Delivery adds a further, quieter problem. Without a system-computed price, courier cost becomes another point of dispute layered on top of an already low-trust transaction, and there is no accepted authority to say whether a quoted fare is reasonable. Meeting in person to avoid that cost carries its own risk, since cash meetings have led to robbery and counterfeit-cash fraud against both buyers and sellers [1, §3.2].

1.3 Objectives

Aim

To design and build a web-based marketplace in which identity verification, escrow-held payment, a fair time-boxed auction mechanism, and delivery coordination work as parts of one settlement flow, for the specific case of individual resale of used goods in Bangladesh.

Specific Objectives

1. Gate the ability to list or transact behind a KYC check, so every seller and buyer in a completed deal is an identity-verified individual, not an anonymous profile.
2. Build an auction mechanism whose end time and highest bid are decided by the server rather than the client, so a fast connection or a manipulated timestamp cannot win an auction unfairly, and add an automatic short extension when a bid arrives near the scheduled close, so a genuine last-second sniper cannot win by timing alone.
3. Route every payment, whether by mobile financial service or card, through a single escrow step that only releases funds to the seller once the buyer confirms receipt, or automatically after a fixed timeout if the buyer does nothing.
4. Compute the distance between buyer and seller automatically and use it either to suggest a safe public meetup point or to book a courier at a fare the system, not either party, has quoted, removing fare negotiation as a point of friction.
5. Keep a readable, auditable record of how each payment was split between seller, courier, and platform, so that a dispute can be resolved by inspecting data rather than by taking someone's word for it.
6. Provide an escrow flow for game-account sales that verifies the seller's control of the account before listing, delivers the login details so that only the buyer can read them, and holds the payment through a protection period after the buyer takes the account over.

1.4 Scope

What the Project Covers

The system is a single responsive web application (no native mobile app) built around one transaction lifecycle: a user verifies their identity, lists or finds an item, buys it outright or wins it at auction, pays into escrow, receives it via self-pickup or courier, and confirms receipt so funds are released. Game accounts follow the same lifecycle, with delivery replaced by a protected credential handover (Section 1.7). Chat, ratings, dispute handling, and notifications exist to support that lifecycle; they are supporting infrastructure, not independent features built for their own sake.

What It Deliberately Excludes

The project is not attempting to be a general e-commerce storefront for new or branded retail, and it is not building a back-office management system. The one internal-facing role in the system, used for KYC review and dispute adjudication, is kept deliberately narrow: giving it general create, edit and delete access over listings, users, or orders would turn the project into exactly the kind of generic admin panel it is trying to avoid being. TrustBazaar does not integrate with game publishers and cannot prevent a publisher from banning a traded account; this is disclosed to buyers before payment. Real financial transactions and production courier bookings are also out of scope for the academic deliverable; sandbox and test-mode integrations stand in for them, and that substitution is disclosed rather than hidden behind a convincing-looking demo.

1.5 Stakeholders

The two primary stakeholders are the platform's users. **Buyers** are identity-verified individuals purchasing used goods or game accounts, either at a fixed price or through the auction mechanism. **Sellers** are identity-verified individuals listing used goods for fixed-price sale or auction, who receive their payout once escrow releases.

The **Platform Operator** is the narrow internal-facing role responsible only for KYC document review and dispute adjudication; the Scope section explains why this role is kept deliberately limited.

Two groups of external providers connect to the system. The **payment providers** are bKash, Nagad and Stripe (card, test mode), whose sandbox APIs the escrow settlement flow integrates with. The **courier providers** fulfil courier-based deliveries through a courier sandbox API where one is available, or through the documented mock courier service otherwise.

Finally, the **development team** (S. M. Nihal Ahmed, Afrim Hossen Khan and Sabikun Nahar Sinthia) is responsible for design, implementation and testing, and the **academic supervisor and evaluators** of the Department of Software Engineering, Daffodil International University, review and assess the capstone deliverable against this proposal and the accompanying SRS.

1.6 Proposed Solution

TrustBazaar is a proposal to close the gap described above for one well-defined slice of the market: individual-to-individual resale of used goods. Rather than trying to be a general marketplace, the project deliberately narrows itself to four problems that are genuinely hard to get right (identity trust, fair auctions, protected payment, and delivery coordination) and treats them as one connected system instead of four separate features bolted together after the fact. Each of these answers a specific loophole identified in the accompanying market analysis [1, §3.6, §4].

A project like this earns its place as a capstone specifically because its interesting parts cannot

be faked with a simple database and a form. First, concurrency has to actually be handled, not assumed away: two people bidding on the same auction in the same second, or two buyers trying to buy the last unit of a fixed-price listing at the same moment, are ordinary situations the system has to get right every single time, not edge cases to patch later once someone complains.

Second, multiple external payment and delivery providers have to agree with each other. A bKash payment, a Nagad payment, and a card payment all have to collapse into the same internal notion of "this order is paid," despite arriving through three different APIs with three different confirmation mechanisms, and a courier's fare quote has to reconcile with what the platform actually charged the buyer.

Third, trust decisions have consequences that compound. A KYC approval, a bid, and a payment release are not independent actions: each one changes what the system is allowed to let happen next, which is a materially different design problem from a typical CRUD workflow where most actions are reversible and independent of each other.

By the end of the project, a demonstrable web application should exist in which a verified user can list an item, another verified user can either buy it or win it in a fair, server-timed auction, pay for it safely, receive it through a system-arranged method, and trust that the money only moved when it was supposed to. The accompanying repository, System Design Document, and test suite should make the reasoning behind each of those guarantees inspectable, not just the working demo itself, so that the project stands up to the question "why should anyone trust this," rather than only to the question "does it work in the demo."

1.7 Game-Account Escrow

Game accounts are sold through the same escrow as physical goods, but because a password cannot be inspected the way a phone can, the handover needs its own sequence of safeguards. Each step is designed to close one of the ways in which today's Facebook-group admin deals fail.

The process begins before anything is sold. When a seller lists a game account, the system issues a one-time code that the seller places in a visible field of the game profile, such as the bio or in-game name, and then uploads a screenshot for the operator to check. This proves that the seller actually controls the account. Once the listing is approved, it is locked to that game's player ID, so the same account cannot be listed or sold twice.

When a buyer purchases the account, the payment goes into escrow first, and only then does the seller hand over the login details. The handover is sealed: the buyer's browser generates an encryption key, the seller's browser encrypts the login details with it, and the server stores only the encrypted result. Neither the platform nor an operator can read the password, which removes the possibility that the middleman, the role an admin plays today, misuses it.

The buyer then has 24 hours to take the account over completely by changing the password, the recovery e-mail and phone number, removing any linked social logins, and turning on two-step verification. If the login details do not work, the buyer reports it within that window and the order goes to a refund review. Once the buyer has confirmed changing the recovery e-mail, however, a later claim that the password did not work is no longer accepted, which prevents a dishonest buyer from keeping both the account and a refund.

Finally, the payment stays in escrow for a seven-day protection period after the takeover. If the seller uses the old e-mail or phone to recover the account during that time, the buyer raises a dispute while the money is still held, so a refund is actually possible. After the money is released, a seller who is found to have recovered an account is suspended and their NID is blacklisted from the platform.

Most game publishers prohibit account transfers and may ban an account that has changed hands [1, §5.2]. For that reason, every game-account listing carries a mandatory disclosure that

the buyer must acknowledge before paying, and the platform's protection is described as covering the handover, the takeover and recovery by the seller during the protection period, not later action by the publisher.

1.8 AI Features

The following AI-assisted capabilities are proposed as enhancements layered on top of the core trust system described above, not as a replacement for any of it: the escrow, KYC, and auction mechanisms must work correctly on their own, and AI features make specific parts of that system smarter or faster to use. Given the project's three-month timeline, each is scoped to use existing pretrained models or managed APIs rather than training a model from scratch, and each is treated as Should-Have or Could-Have priority (see the Timeline and Work Plan), so the Must-Have trust guarantees are never traded off to make room for them.

Fraud and Anomaly Detection on Bidding Patterns

The auction engine (FR-AUC-01–FR-AUC-18) logs every bid with its bidder, amount, and timestamp, data that already exists for the concurrency-correctness reasons described in System Requirements. This feature adds a lightweight anomaly-scoring pass over that log to flag suspicious patterns, such as a cluster of small proxy bids from newly created accounts driving up a price, or a bidder who consistently bids only in the final seconds across many auctions. Flagged activity surfaces to the Platform Operator's dispute and review queue rather than auto-blocking a bid, since a false positive here should never silently cost a legitimate bidder an auction they fairly won.

AI-Assisted KYC Document Verification

The KYC module (FR-KYC-01–FR-KYC-12) currently routes every submitted document to manual operator review. This feature adds an automated pre-check step before that review, verifying that an uploaded ID photo is legible and correctly cropped and that the selfie plausibly matches the ID photo, using an existing document-verification or face-match API rather than a custom-trained model. This does not replace the operator's approval; it only filters out unusable submissions (blurry photos, mismatched selfies) before they reach the review queue, reducing manual review load without changing who has final approval authority.

Listing Recommendation Engine

Once enough listings and purchase history exist, a recommendation engine can surface listings a buyer is more likely to want on the browse and search page (FR-SRC-01–FR-SRC-06), using a similarity-based approach (category, price range, and past interaction history) rather than a deep learning model that would need far more data than a three-month, single-market pilot can realistically generate. This is explicitly a Could-Have: it improves discovery, but the platform is fully usable through search and filtering alone without it.

Auto-Generated Listing Descriptions from Photos

To lower the friction of creating a listing (FR-LST-01–FR-LST-13), a seller could optionally have a draft title and description generated from their uploaded photos via an image-captioning or vision-language API, which they review and edit before publishing. It is never auto-published without the seller's confirmation, since an inaccurate AI-generated description of a used item's condition would directly undermine the trust the rest of the platform is built around.

Scope Note

All four features depend on the underlying data (bids, KYC documents, listings, photos) that the core system already produces, so they are additive integrations rather than a parallel system, and each can be descoped independently if time runs short without affecting the Must-Have deliverable described in Section 1.6.

2 System Requirements

The full requirements are specified in the accompanying Software Requirements Specification (`docs/srs.tex`, version 2.0, IEEE 830-1998 format). It contains 191 functional requirements in 17 areas, 86 security requirements and 41 other non-functional requirements, each with a MoSCoW priority and a verification method, together with the configurable parameters (time limits, thresholds and fees) that the requirements refer to. Each functional area has its own identifier prefix, such as FR-AUC for the auction engine, and maps to one backend module. This section summarizes that specification at the level relevant to project approval, rather than reproducing it.

2.1 Functional Requirement Areas

The requirements follow the transaction lifecycle. A user first registers and signs in (FR-ACC) and passes KYC verification (FR-KYC), after which they can create fixed-price or auction listings (FR-LST) and find items through search and discovery (FR-SRC). Purchases then take one of two paths: the auction engine, with server-authoritative bidding, proxy bids, a reserve price and anti-sniping (FR-AUC), or fixed-price purchase with a concurrency-safe stock reservation (FR-FIX).

Every purchase is settled through payment and escrow (FR-PAY), including seller payouts and refunds, and fulfilled either through the delivery area (FR-DLV), which chooses between a safe meetup with a handover code and a courier, or, for game accounts, through the game-account escrow (FR-GAM), which adds ownership proof, sealed credential handover, guided takeover and the protection hold described in Section 1.7. Around this core sit the supporting areas: real-time chat (FR-CHT); ratings, reviews and trust signals (FR-REV); dispute resolution (FR-DSP); and notifications (FR-NTF). Monetization, covering the platform commission and delivery margin, is specified in FR-MON; trust and safety controls in FR-TNS; the deliberately narrow operator and administrator functions in FR-OPS; and privacy and data-subject rights in FR-PRV. Security requirements (SEC) are collected in a separate chapter of the SRS, based on OWASP ASVS 5.0 Level 2.

2.2 Key Non-Functional Requirements

The requirements that most directly shape the architecture are the ones concerning correctness under concurrency and real-time state propagation. The auction engine's end time and highest bid must be authoritative on the server regardless of client-side timestamps or connection speed; live bid and chat updates must propagate within approximately one second; and the system must run as separate, independently restartable containers (the backend API with its real-time gateway, the web client, the database and the cache), so that no single process holding in-memory state can silently corrupt an in-progress auction.

2.3 Acceptance Criteria

Rather than judging the project by feature count, the following properties are treated as the actual deliverable, since the rest of the system exists to give them a context to operate in:

1. Two simultaneous bids on the same auction never both register as the winning bid.
2. Two simultaneous buyers can never both purchase the last unit of a fixed-price listing.
3. An escrow payment is never released to a seller without either explicit buyer confirmation or a genuine timeout.
4. A payment webhook delivered twice by a provider never results in the order being charged or settled twice.
5. Game-account login details are never stored on the server in a form the server or an operator can read.
6. Payment for a game-account order is never released before the protection period ends, unless the buyer releases it early or an operator decides a dispute.

These are verified by an automated test suite that specifically targets simultaneous bidding, simultaneous stock decrement, repeated or duplicate payment-webhook delivery, the correctness of the fee-split arithmetic, and the game-account release rules. The suite runs automatically on every push and pull request to the main branch, so a regression is caught before it ever reaches a demonstration.

3 Tools and Technologies

The technology stack below is the one set up in the project repository [2], which already contains the module structure, data model, container setup and continuous-integration pipeline that the implementation will build on. Each choice is made for a reason specific to this project rather than by default.

3.1 Backend

The backend is a **NestJS 10** application written in **TypeScript 5** and running on **Node.js 20**. NestJS was chosen because its module system maps directly onto the SRS: the repository has one module per feature area (authentication, users, KYC, listings, auctions, orders, payments, delivery, chat, reviews, disputes and notifications), so each requirement range has an obvious home and the three team members can work on separate modules without stepping on each other. Every incoming request passes through a global validation layer built on **class-validator** and **class-transformer**, which checks the request against its expected shape and rejects any fields the endpoint does not expect, so malformed or over-posted data never reaches the business logic.

Users will authenticate with **JSON Web Tokens**, issued through **@nestjs/jwt** and checked by **Passport** with the **passport-jwt** strategy, and passwords will be stored only as **Argon2id** hashes, which replace the **bcrypt** library currently in the project because bcrypt ignores input beyond 72 bytes and would truncate long Bangla passphrases. Session tokens are carried in `HttpOnly` cookies rather than in browser storage, so injected scripts cannot read them. Configuration such as secrets, database addresses and provider keys is loaded from environment variables through **@nestjs/config**, so no credential is ever written into the code.

3.2 Data Storage

PostgreSQL 15 is chosen over a document store because settlement and bidding correctness genuinely depend on transactional (ACID) guarantees; this is a case where that property is load-bearing, not a formality that could be swapped out later. The application reaches it through the **TypeORM 0.3** object-relational mapper and the **pg** driver. The data model is already defined as fourteen entities covering users, KYC documents, listings, bids, orders, escrow transactions, the settlement ledger, delivery requests, chat threads and messages, reviews, disputes, featured listings and subscriptions. The escrow transaction records its state (held, released, refunded, or frozen under dispute) together with an append-only history of every change, which is what allows a dispute to be decided by inspecting the record rather than by taking either party's word. Automatic schema synchronisation is used only during early development and will be replaced by versioned migrations before any real data exists.

Redis 7, accessed through the **ioredis** client, holds auction countdown state and coordinates bid locks outside any single server process's memory, so restarting or scaling the backend cannot accidentally reset or extend a live auction's true end time. The same Redis instance provides publish/subscribe messaging between server instances and the counters used for rate limiting.

3.3 Real-Time Communication

Live bid updates, countdown synchronisation, chat delivery and notifications are carried over **Socket.IO 4** through a NestJS WebSocket gateway, since a marketplace where the displayed highest bid is stale by several seconds undermines the fairness the auction engine is meant to guarantee in the first place. Each auction and each chat thread will be its own room, so an accepted bid is broadcast only to the people watching that listing; the gateway and its bid-broadcast method are already in place. When more than one backend instance is running, the gateway relies on Redis publish/subscribe rather than keeping connection state in a single process's memory.

3.4 Frontend

The web client is built with **React 18** and **TypeScript**, using **React Router 6** for navigation and **Vite 7** as the development server and build tool. The client talks to the backend through a small API layer that sends each request with the session cookie set by the backend. The first screens set up in the repository are the home page, login, the KYC submission wizard and the listing detail page, which together cover the entry point of the transaction lifecycle. For the game-account escrow, the browser's built-in **Web Crypto API** will perform the key generation and encryption of login details on the client, so the server never handles a readable password and no custom cryptography is written.

3.5 External Integrations

Payments use each provider's sandbox or test mode: the **bKash** tokenized-checkout sandbox, the **Nagad** merchant sandbox, and **Stripe** in test mode for cards, with Stripe's webhook signing secret used to verify that payment confirmations genuinely come from the provider. For delivery, a single configuration setting selects the courier provider: a documented **mock courier** by default, or the **Pathao** or **Steadfast** courier APIs where sandbox access is available. Where no public courier sandbox exists, the mock service is used and clearly labelled as such rather than presented as a live integration.

3.6 Infrastructure, Testing and Quality

Docker Compose runs the system as four separate containers (PostgreSQL, Redis, the backend and the frontend) built on lightweight Alpine Linux images. Health checks make the backend wait until the database and cache are actually ready before it starts, and the arrangement mirrors how the pieces would be deployed, keeping local development close to that reality rather than diverging from it.

Tests are written with **Jest 29** and **ts-jest**. Alongside ordinary unit tests, the repository already defines a dedicated concurrency test suite whose cases correspond to the acceptance criteria: simultaneous bidding, the anti-sniping extension, proxy bidding, the last-unit purchase race, duplicate payment webhooks and fee-split reconciliation. Each case is currently marked as pending and is filled in as the feature behind it is built, and a feature is not considered finished until its case passes. Code style and common errors are checked by **ESLint** with the TypeScript rules.

Every push and pull request to the main branch triggers a **GitHub Actions** pipeline that lints the backend, runs the unit tests, and runs the concurrency suite against real PostgreSQL and Redis service containers, then builds the frontend, so a regression is caught before it is merged. **GitHub CodeQL** scans the TypeScript code for security weaknesses on every change and again weekly, and **Dependabot** proposes weekly minor and patch updates to dependencies, while major-version upgrades are held back for deliberate, manual handling.

4 Project Timeline and Work Plan

4.1 Development Approach

The build proceeds as a sequence of working, demonstrable increments rather than a single delivery at the end. Each increment adds one complete slice of the transaction lifecycle (data model, backend logic, and the corresponding UI together), so that at every stage there is something that can actually be clicked through, not just a partially wired backend sitting behind a placeholder screen.

Planned Increments

The first increment establishes identity and accounts: registration, login, and the KYC submission-and-review path that gates everything downstream. The second adds listings, covering the creation of fixed-price and auction listings and the browsing, searching and filtering of the catalogue by category, price, condition and distance.

The third and fourth increments build the two purchase paths. The auction path brings in server-authoritative bidding, automatic proxy bids, the reserve price and the anti-sniping extension, tested specifically under simultaneous bidders rather than only one bidder at a time. The fixed-price path adds purchase with a stock decrement that stays correct when several buyers try to claim the last unit at once.

The fifth increment connects payment and escrow, integrating the mobile financial services and card processor sandboxes with the hold-then-release settlement logic behind both. The sixth adds delivery: the distance computation and the self-pickup or courier decision, using a courier sandbox or its documented mock substitute. The seventh builds the trust and communication layer of chat, ratings, disputes and notifications, wired to the specific transactions they support rather than built as generic messaging.

The eighth increment adds the game-account escrow (ownership proof, sealed credentials, guided takeover and the protection hold), built on the escrow and dispute logic already in place. The ninth and last covers monetization and hardening: the commission and delivery-margin

calculation on top of the existing ledger, followed by a pass on rate limiting, audit logging, and behaviour under concurrent load.

4.2 Team and Roles

The project is undertaken by a three-member team: S. M. Nihal Ahmed (ID: 232-35-002), Afrim Hossen Khan (ID: 232-35-076) and Sabikun Nahar Sinthia (ID: 232-35-475).

Having three people changes the methodology in one important way: instead of the strictly sequential, one-slice-at-a-time build described for a solo effort, the workstreams below are split across team members and run partly in parallel once the shared foundation (data model and authentication) is in place, which is what makes a three-month timeline realistic rather than optimistic. A shared Git repository with feature branches and a lightweight code-review step before merging to main keeps the three workstreams from silently drifting apart. Development, sandbox accounts, and hosting are expected to run on free or student-tier offerings (payment provider sandboxes, a free-tier cloud instance or local Docker Compose setup for demonstration), keeping the resource footprint of the build close to zero cost.

4.3 Timeline (12 Weeks / 3 Months)

The three core workstreams (the auction and listings path, the payment and escrow path, and the delivery and trust-layer path) are assigned to the three team members and developed in parallel from Week 3 onward, against the shared data model and authentication built together in Weeks 1–2.

Weeks	Focus	Owner(s)
1–2	Shared setup: repo, Docker Compose, data model, auth, KYC flow	All three, together
3–5	Auction engine, fixed-price purchase, concurrency tests	Member A
3–5	Payment/escrow integration, settlement ledger	Member B
3–5	Listings, search/discovery, delivery decision engine	Member C
6–7	Integrate the three workstreams into one order lifecycle	All three, together
8	Chat and notifications	Member A
8	Game-account escrow: sealed credentials, ownership proof	Member B
8	Ratings and disputes	Member C
9	Monetization features (commission, delivery margin)	Member A
9	Game-account escrow: guided takeover, protection hold	Member B
9	Security hardening: rate-limiting, audit logging, IDOR checks	Member C, reviewed by all
10	End-to-end and load testing of concurrency-sensitive paths	All three, together
11	Bug fixing, System Design Document, README	All three, together
12	Final polish, demonstration rehearsal, submission	All three, together

Member assignment (A/B/C) is deliberately left as a role rather than pinned to a specific name here, so the team can assign it based on who ends up strongest in each area during Weeks 1–2. The auction engine and the payment and escrow flow remain the critical path, since chat, ratings, disputes, monetization and the game-account escrow all depend on an order existing and having been paid for correctly, so those two workstreams get first claim on time if Weeks 3–5 run long. If the schedule slips, the features scoped down first are the Should-Have and Could-Have ones (delivery automation, notifications, featured listings, the Pro Seller tier), since the Must-Have core (KYC, the auction engine, escrow settlement, chat, disputes and the game-account escrow) is what the project's central claim depends on.

4.4 Risk Management

The project carries five main risks, each with a planned response.

Sandbox access delays. Developer sandbox approval for bKash and Nagad can be slow to obtain. If it is not available in time, Stripe test mode alone will be used, and the limitation will be stated openly in the final documentation rather than worked around silently.

No public courier sandbox. If neither Pathao nor Steadfast sandbox access is available, the mock courier service, with realistic fare and status behaviour, stands in for them and is documented explicitly as a substitution rather than presented as if it were live.

Concurrency defects. This is the single biggest technical risk. It is mitigated by row-level locking in PostgreSQL and by writing the concurrency tests before the corresponding feature is considered finished, not after a bug is reported; the cases of the concurrency suite are already defined in the repository and run in continuous integration on every change.

Publisher terms on game accounts. Publishers may ban accounts that have been traded [1, §5.2]. The response is threefold: buyers must acknowledge a mandatory disclosure before paying, the platform's protection is limited to the handover and the protection period, and the game-account listing type sits behind a configuration switch, so it can be disabled without affecting the rest of the platform if that becomes necessary.

Account recovery after release. An original owner may still recover a sold account through the publisher's own support process, which no marketplace can fully prevent. The seven-day protection hold reduces this risk, and identity-based accountability for the seller deals with it if it happens after the money has been released.

References

[1] S. M. N. Ahmed, A. H. Khan and S. N. Sinthia, "Trust in Bangladesh's Second-Hand Online Market: A Market Analysis of Bikroy.com and Its Alternatives," supplementary report to the TrustBazaar project proposal, Department of Software Engineering, Daffodil International University, October 2026.

[2] TrustBazaar project repository. `https://github.com/nihal4/trustbazaar` (accessed 7 Oct 2026).